

BSc (Hons) in **Computer Science****SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing**Practical 01**

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation**Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P - Performance Measure: The criteria used to evaluate the agent's success or utility (e.g., maximizing an expected outcome or objective metric). E - Environment: The world or conditions in which the agent operates. A - Actuators: The mechanisms the agent uses to perform actions and alter its state within the world. S - Sensors: The

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.opponents`: Represents the dynamic, moving physical adversaries in the grid.

`self.agent_pos`: Represents the physical coordinate location of the agent itself within

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

are not static obstacles like walls. They dynamically execute their own actions (moving Up, Down, Left, Right, or Stay) at every time step.

Direct Interaction: The actions of the opponents directly alter the environment state.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

The Core Mechanism: AI is formally defined (via Russell & Norvig) as the study of agents that "receive percepts from the environment and perform actions". The Continuous Sequence: Instead of taking a single, static snapshot, an intelligent agent establishes an "ongoing structural loop" with its environment. As it acts, it

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

In the terminology of Artificial Intelligence and the PEAS framework, Sensors are the specific mechanisms that allow an agent to receive input. Since the `get_percept()` method actively gathers the current state of the environment (like a camera, radar, or human sensor would in the real world) and feeds that "percept" back to the agent, it is

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Global Food Locations: The agent knows how many food items are left (`remaining_food`), but it does not receive the `self.food_positions` set. It must blindly search the grid until it steps on one (`smells_food`).

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

In the code, hitting a wall triggers `self.score -= 5`. As discussed in your lecture materials, the Performance Measure is the criteria used to evaluate how successful the agent is at achieving its objective. The score variable serves as this exact metric,

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

measurability and objectivity

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Explain the shift in observability...

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Relate the new sensor data to rational decision-making...

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

Recall the vacuum world exploit...

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING